Università degli Studi di Salerno

Dipartimento di Ingegneria dell'Informazione ed Elettrica e Matematica Applicata

Corso di Laurea Magistrale in Ingegneria Informatica

Algoritmi e Protocolli per la Sicurezza
Canale I-Z

Project Work

Gruppo **ADAM**

WP	Cognome e Nome	Matricola	e-mail	Responsabile
1	Lettieri Alessia	0622702324	a.lettieri36@studenti.unisa.it	WP3
2	Napolitano Domenico	0622702325	d.napolitano18@studenti.unisa.it	WP2
3	Panico Marco	0622702416	m.panico20@studenti.unisa.it	WP4
4	Passanante Alessandro	0622702333	a.passannante1@studenti.unisa.it	WP1

Anno accademico 2023-2024

Sommario

WP1 ..... ***4***

1.1 Attori del sistema **4**
Utenti (Client) 4
Autorità di Certificazione (IPZS) 4
Autorità di Credenziali 5
Servizi Qualificati (Server) 5

1.2 Threat model **5**
Impersonator 5
Malicious Authority 5
Dinosaurs (Opponents of innovation) 6
The Eavesdropper 6
Internal opponent 6
Colluding Users 7
User Requesting Corrupt Entity Credentials 7
Expired ID User 7

1.3 Proprietà di Sicurezza **8**
Confidenzialità 8
Integrità 8
Trasparenza 8
Efficienza/Usabilità 9

1.4 Completeness **9**

WP2 ..... ***10***

2.1 Panoramica Generale di Funzionamento **10**

2.2 Supposizioni **10**

2.3 Identificazione dell'utente e rilascio credenziali **11**
2.3.1 Rilascio credenziali 12
Credenziali 13

2.4 Accesso ai servizi **13**

2.5 Specifiche utilizzate **14**

WP3 ..... ***17***

3.1 Confidenzialità **17**

3.2 Integrità **18**

3.3 Trasparenza **19**

3.3 Efficienza/Usabilità **20**

3.4 Valutazioni **20**

3.5 Contrasto degli avversari **22**
Impersonator 22
Malicious Autority 23
Dinosaurs 23
The Eavesdropper 24
Internal Opponent 24
Colluding Users 24
User Requesting Corrupt Entity Credentials 24
Expired ID User 25
Conclusioni 25

WP4 ..... *26*
4.1 Init 27
4.2 certificato_utente 27
4.3 certificato_autorità 29
4.4 certificato_autorità2 30
4.5 richiesta_rilascio_credenziali 30
4.6 richiesta_rilascio_credenziale2 33
4.7 accesso_servizio 33
4.8 accesso_servizio_2cred 36

WP1

In questo capitolo iniziale, ci focalizzeremo sulla definizione degli attori onesti che partecipano al sistema, esaminando i loro scopi e le funzionalità che intendono implementare. Gli attori onesti sono coloro che operano nel rispetto delle regole e delle politiche stabilite, cercando di raggiungere i propri obiettivi senza compromettere l'integrità del sistema.

Poi, analizzeremo i possibili avversari (o threat models) che potrebbero tentare di compromettere il sistema, valutando le loro risorse e le motivazioni che li spingono ad agire. Questa analisi ci consentirà di identificare i tipi di attacchi che il sistema potrebbe subire e di determinare le misure di sicurezza necessarie per contrastarli.

Dopo aver compreso il contesto operativo del sistema, identificheremo le proprietà di sicurezza che devono essere mantenute anche in presenza di attacchi. Queste proprietà sono essenziali per assicurare il corretto funzionamento del sistema e la protezione delle informazioni sensibili.

1.1 Attori del sistema

In questo scenario, sono coinvolti diversi attori nel sistema, ciascuno con obiettivi e funzionalità specifiche. Di seguito viene fornito un elenco dettagliato di tali attori e delle loro responsabilità.

Utenti (Client)

- Descrizione: Persone che richiedono accesso a servizi qualificati.
- Ruoli:
    1. Ottengono e conservano la propria Carta d'Identità Elettronica (CIE).
    2. Richiedono credenziali specifiche dalle autorità competenti.
    3. Utilizzano le credenziali per accedere ai servizi qualificati offerti dai server.

Autorità di Certificazione (IPZS)

- Descrizione: Entità che rilascia la carta di identità elettronica (CIE)
- Ruoli:
    1. Rilascia le CIE contenenti certificati X509v3.
    2. Verifica l'identità degli utenti al momento del rilascio della CIE.
    3. Garantisce la validità e l'autenticità delle CIE e dei relativi certificati.
    4. Implementa il sistema di firma all'interno della CIE.

Autorità di Credenziali

- Ruolo: Entità che emettono credenziali specifiche per vari requisiti (es. residenza, luogo di nascita).
- Responsabilità:
    1. Verificare i dati dell'utente attraverso i propri database.
    2. Rilasciare credenziali.
    3. Mantenere la validità e la sicurezza delle credenziali emesse.

Servizi Qualificati (Server)

- Ruolo: Sistemi che offrono servizi ristretti ai possessori di specifiche credenziali.
- Responsabilità:
    1. Verificare le credenziali presentate dagli utenti per concedere l'accesso ai servizi;
    2. Riconoscere e fidarsi delle autorità di rilascio delle credenziali;
    3. Definire politiche di accesso basate sulle credenziali possedute dagli utenti

1.2 Threat model

Nel progettare un sistema di identificazione basato su credenziali, è importante considerare i diversi avversari che potrebbero tentare di attaccarlo. Questi avversari possono agire da soli o in gruppi, cooperando per raggiungere i loro scopi. Comprendere queste minacce è essenziale per creare un sistema sicuro, efficiente e trasparente per tutti gli utenti.

Impersonator

- Tipologia: Attivo
- Descrizione: Individuo che cerca di impersonare un utente legittimo utilizzando le sue credenziali rubate o falsificate.
- Risorse: Possiede una discreta potenza di calcolo e accesso a informazioni personali dettagliate.
- Attacchi:
    - Impersonificazione: Utilizzo di credenziali rubate per accedere ai servizi come se fosse l'utente legittimo.
    - Falsificazione: Creazione di credenziali false per aggirare i controlli di accesso.

Malicious Authority

- Tipologia: Attivo

- Descrizione: Autorità che emette credenziali false o manipolate.
- Risorse: Accesso a risorse significative e capacità di emettere credenziali.
- Attacchi:
    - Emissione di Credenziali False: Rilascio di credenziali fraudolente per utenti non autorizzati.
    - Manipolazione dei Dati: Alterazione delle informazioni nelle credenziali per ingannare il sistema.

Dinosaurs (Opponents of innovation)

- Tipologia: Passivo
- Descrizione: Oppositori dell'innovazione, che anziché studiare le nuove tecnologie non fanno altro che indicarne genericamente i rischi con il solo scopo di lasciare tutto così com'è. Sono in genere allarmisti che puntano sul fatto che tutti i dispositivi possono essere compromessi, la sicurezza assoluta non esiste. Quindi, possono sollevare preoccupazioni riguardanti la sicurezza e la privacy del sistema e devono essere considerati nella progettazione per mitigare le loro potenziali obiezioni.
- Risorse: Spiccate abilità comunicative, capacità di influenzare l'opinione pubblica.
- Attacchi:
    - Campagne di Disinformazione: Diffusione di notizie negative per ostacolare l'adozione del sistema.
    - Lobbying: Pressioni su enti regolatori per bloccare o ritardare l'implementazione del sistema.

The Eavesdropper

- Tipologia: Passivo
- Descrizione: Avversario interessato a intercettare tutte le operazioni del sistema. Può essere sia interno all'organizzazione che esterno e non ha come principale obiettivo quello di compromettere il normale funzionamento del sistema, ma semplicemente è interessato a raccogliere quante più informazioni possibili.
- Risorse: Potenza computazionale non trascurabile, accesso alle varie linee di comunicazione tra le parti del sistema.
- Attacchi:
    - Intercettazione delle Comunicazioni: Utilizzando tecniche di intercettazione di rete, può raccogliere dati sensibili scambiati tra gli utenti e il server.
    - Analisi del Traffico: Monitorando il volume e le caratteristiche del traffico, può dedurre informazioni sensibili anche senza accedere al contenuto dei messaggi.

Internal opponent

- Tipologia: Attivo

- Descrizione: Individuo interno al sistema che tenta di acquisire in modo improprio le informazioni personali degli utenti.
- Risorse: Accesso a risorse interne e conoscenza dei sistemi e delle politiche.
- Attacchi:
    - Furto di Dati: Appropriazione indebita di informazioni sensibili dall'interno.

Colluding Users

- Tipologia: Attivo
- Descrizione: Gruppo di utenti che collabora per combinare le proprie credenziali al fine di accedere a servizi che richiedono più credenziali.
- Risorse: Maggiori risorse computazionali rispetto a un singolo avversario e capacità di coordinazione.
- Attacchi:
    - Combining Credentials: Unione delle credenziali di diversi utenti per soddisfare i requisiti di accesso ai servizi.

User Requesting Corrupt Entity Credentials

- Tipologia: Attivo
- Descrizione: L'ente corrotto si presenta come un'autorità fidata per ingannare un utente e ottenere informazioni sensibili. Una volta ottenute queste informazioni, l'ente richiede le credenziali a un'autorità legittima, fingendo di essere l'utente. In questo modo, l'ente agisce come intermediario fraudolento, ottenendo accesso a risorse in maniera illecita e compromettendo la sicurezza degli utenti e dei sistemi coinvolti.
- Risorse: Possiede una discreta potenza di calcolo ed ha accesso al canale di comunicazione tra utente e autorità, oltre a possedere informazioni personali dettagliate.
- Attacchi:
    - Richiesta di Credenziali Fraudolente: Ottenimento di credenziali in maniera illecita.

Expired ID User

- Tipologia: Attivo
- Descrizione: Un individuo che tenta di ottenere credenziali utilizzando una carta d'identità scaduta.
- Risorse: Possiede una carta d'identità scaduta.
- Attacchi:
    - Dead ID: Utilizzo diretto della carta d'identità scaduta per richiedere credenziali presso l'Autorità di Credenziali.

1.3 Proprietà di Sicurezza

Le principali proprietà di sicurezza che il sistema deve possedere per garantire un ambiente sicuro e affidabile sono analizzate in base ai quattro pilastri fondamentali: confidenzialità, integrità, trasparenza ed efficienza.

Confidenzialità:

- **C.1**: Le credenziali degli utenti devono essere protette e accessibili solo dalle parti autorizzate in maniera controllata.
- **C.2**: Le comunicazioni tra gli utenti e i servizi devono essere protette per garantire che nessuna informazione sensibile venga intercettata.
- **C.3**: Le informazioni personali degli utenti devono essere minimizzate nelle comunicazioni per evitare esposizioni non necessarie.
- **C.4**: Il sistema deve essere progettato in modo che un utente malintenzionato non possa usufruire dei servizi disposti, identificandosi con le credenziali di un altro utente.
- **C.5**: Le credenziali devono poter essere mostrate in maniera sicura e verificabile, garantendo che tali informazioni, una volta verificate per l'autenticazione, non vengano conservate o memorizzate dal server per qualsiasi scopo.

Integrità

- **I.1**: Le credenziali devono essere verificabili e non falsificabili, garantendo che siano state emesse da un'autorità riconosciuta.
- **I.2**: Le politiche di accesso devono essere applicate correttamente, impedendo l'accesso a chi non possiede le credenziali richieste.
- **I.3**: La chiave segreta della CIE deve rimanere nascosta e non accessibile a nessun attaccante.
- **I.4**: Il sistema deve garantire che le credenziali non possano essere combinate da utenti diversi per accedere a servizi riservati a chi possiede entrambe le credenziali.
- **I.5**: Le informazioni scambiate tra gli utenti e il server devono mantenere la loro integrità durante tutto il processo di comunicazione, evitando modifiche non autorizzate.

Trasparenza:

- **T.1**: Gli algoritmi e i protocolli utilizzati per la gestione delle credenziali e l'accesso ai servizi devono essere pubblicamente noti e verificabili.
- **T.2**: Il processo di emissione, verifica e gestione delle credenziali deve essere trasparente e verificabile da parte degli utenti e delle autorità competenti.
- **T.3**: Minimizzare l'affidamento su terze parti fidate, garantendo che il sistema sia progettato per essere verificabile indipendentemente.

Efficienza/Usabilità:

- **E.1**: Il sistema deve garantire tempi di risposta rapidi per la verifica delle credenziali e l'accesso ai servizi.
- **E.2**: Il processo di autenticazione dell'utente, tramite CIE deve essere rapido e sicuro, minimizzando i tempi di attesa per l'utente.
- **E.3**: L'autorità di credenziali deve garantire che il rilascio della credenziale sia eseguita in modo rapido ed efficiente, riducendo al minimo l'impatto sulle prestazioni del sistema e l'esperienza degli utenti.

1.4 Completeness

Il sistema proposto mira a garantire l'accesso remoto ai servizi solo a coloro che possiedono le credenziali richieste, seguendo determinate politiche di accesso.

All'istante T0 tutti gli utenti ottengono la propria CIE (carta d'identità elettronica) e il PIN.

All'istante T1 l'utente si identifica con l'autorità competente e richiede il rilascio di una certa credenziale con lo scopo di accedere a un servizio.

All'istante T2 l'autorità accede ai database e in caso di esito positivo rilascia la credenziale richiesta dall'utente.

All'istante T3 l'utente richiede un servizio qualificato sfruttando le credenziali fornite dall'autorità competente.

All'istante T4, se le credenziali fornite sono valide, l'utente può utilizzare il servizio richiesto.

WP2

In questo capitolo ci concentreremo sulla presentazione di una soluzione che risponde al modello identificato nel Work Package 1 (WP1). L'obiettivo è quello di proporre un sistema che riesca a raggiungere un ragionevole compromesso tra efficienza, trasparenza, confidenzialità e sicurezza.

2.1 Panoramica Generale di Funzionamento

In questa sezione, verrà descritto il processo di generazione, rilascio e utilizzo delle credenziali necessarie per accedere ai servizi remoti. Le credenziali sono emesse da autorità fidate e sono necessarie per garantire che solo utenti autorizzati possano accedere ai servizi qualificati. Questo processo coinvolge diversi attori chiave, tra cui l'utente, le autorità di rilascio delle credenziali, e il server dei servizi.

Il processo di generazione delle credenziali inizia con la richiesta da parte dell'utente, che utilizza la propria CIE per autenticarsi. La CIE è un documento elettronico sicuro che contiene una chiave segreta utilizzata per firmare digitalmente le richieste. Una volta che la richiesta è stata firmata e inviata all'autorità di rilascio delle credenziali, questa verifica l'autenticità della richiesta e rilascia, se l'utente è idoneo, le credenziali necessarie, firmate digitalmente per garantirne l'integrità e l'autenticità.

Una volta ottenute, le credenziali possono essere utilizzate dall'utente per accedere ai servizi remoti. Il server dei servizi verifica le credenziali ricevute confrontandole con le politiche di accesso definite. Solo le credenziali valide e autentiche permetteranno l'accesso ai servizi richiesti. L'intero processo è progettato per essere sicuro, efficiente e rispettoso della privacy dell'utente.

2.2 Supposizioni

Nel contesto del progetto, si fanno diverse supposizioni per garantire il funzionamento e la sicurezza del sistema. Innanzitutto, l'Istituto Poligrafico e Zecca dello Stato (IPZS) e l'Anagrafe sono considerati come parti fidate.

Ogni utente riceve la Carta d'Identità Elettronica (CIE) e il relativo PIN direttamente dall'IPZS o dall'Anagrafe, e si presume che la CIE contenga informazioni corrette e affidabili. Il PIN è una stringa di 10 bit non coincidente con la chiave pubblica presente nel certificato digitale.

La CIE include una chiave segreta di firma e un certificato digitale di tipo X509v3. Questa configurazione permette agli utenti di ottenere una firma digitale interrogando la CIE con una query SIGN(PIN, hash), a patto che il PIN fornito sia corretto.

Si confida inoltre che le autorità incaricate di rilasciare le credenziali seguano procedure sicure e operino con onestà durante l'emissione delle credenziali stesse. Quindi, supponiamo che anche le autorità di rilascio credenziali possiedano il certificato digitale in formato X509v3, firmato anch'esso dall'IPZS, in quanto parte fidata.

Le credenziali sono emesse digitalmente e firmate dall'autorità emittente, garantendo così la loro autenticità.

L'ente che fornisce l'accesso ai servizi è sempre un attore onesto e quando riceve le credenziali da un utente non le conserva.

Per quanto riguarda la sicurezza dei dispositivi degli utenti, si assume che i dispositivi di coloro che operano onestamente non siano compromessi. Questo implica che né l'hardware né il software dei loro dispositivi siano stati infettati o controllati da malware, virus, o trojans, mantenendo così l'integrità del sistema.

2.3 Identificazione dell'utente e rilascio credenziali

L'identificazione dell'utente nel nostro sistema avviene mediante l'utilizzo della Carta d'Identità Elettronica (CIE) e un protocollo di connessione sicura basato su TLS (Transport Layer Security). Ecco come avviene il processo:

I. **Connessione sicura TLS**: Quando un utente desidera accedere ai servizi del sistema, viene stabilita una connessione sicura tramite il protocollo TLS tra il dispositivo dell'utente e il server. Questo garantisce che tutte le informazioni scambiate siano crittografate e protette da eventuali intercettazioni o manipolazioni da parte di terzi.

II. **Autenticazione con la CIE**: L'utente inserisce il proprio PIN associato alla CIE. Il PIN è una stringa di 10 bit che funge da chiave per accedere alla CIE e confermare l'identità dell'utente.

III. **Verifica dell'autenticità**: Il sistema invia la richiesta di firma digitale alla CIE dell'utente, insieme al PIN inserito. La carta d'identità elettronica include una chiave segreta di firma e un corrispondente certificato digitale (il tipo scelto è X509v3); sebbene quest'ultimo sia leggibile accedendo alla CIE, la chiave resta nascosta. La CIE, se il pin corrisponde a uno specifico PIN memorizzato, attraverso la query SIGN(pin, hash), restituisce una firma ECDSA del messaggio m di cui hash ne rappresenta lo SHA256. Altrimenti restituisce un errore.

    Inoltre, il certificato possiede la data di scadenza (campo "not valid after"), quindi l'autorità deve verificare se è valido in quel momento, altrimenti rifiutare la richiesta per le credenziali.

IV. **Rilascio credenziali**: Una volta confermata l'autenticità dell'utente, l'autorità rilascia le credenziali per i servizi richiesti. Questo argomento verrà dettagliato successivamente nel paragrafo 2.3.1.

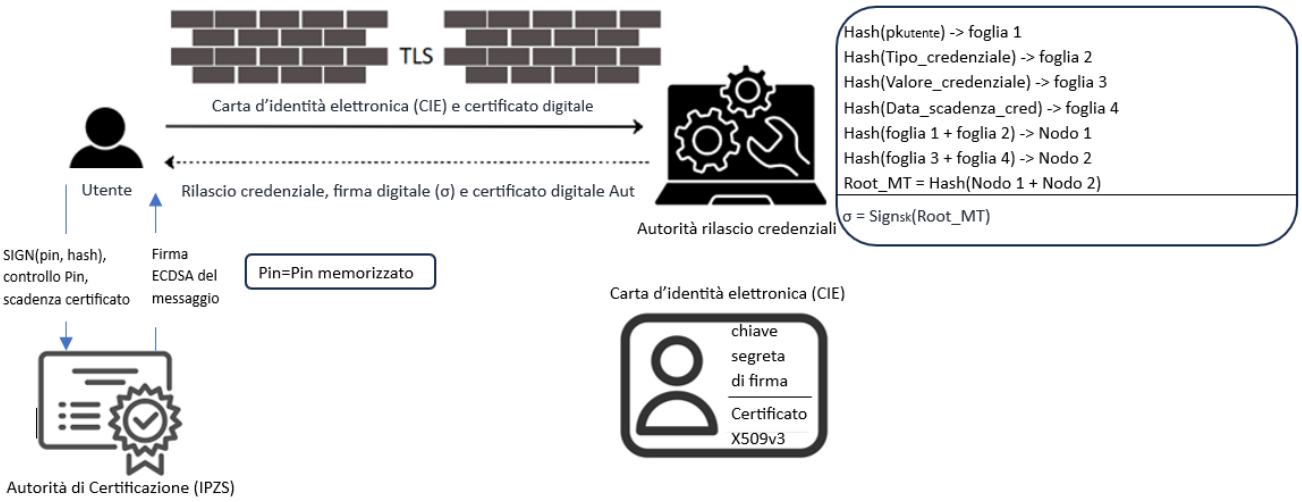

Figura 1 Identificazione dell'utente e rilascio credenziali

2.3.1 Rilascio credenziali

L'Autorità di Credenziali implementa un sistema basato su Merkle Tree per rilasciare e verificare le credenziali degli utenti in modo sicuro ed efficiente, integrando anche il concetto di commitment delle foglie per migliorare ulteriormente la sicurezza e l'integrità del processo.

Quando un utente richiede una credenziale, l'Autorità verifica l'identità dell'utente tramite la sua CIE, che contiene la chiave pubblica e il certificato digitale rilasciato dall'IPZS. Si può quindi fare affidamento al fatto che l'utente conosce un'informazione segreta (sk) correlata a tale chiave pubblica. L'esito della verifica viene comunicato all'utente e in caso esso sia positivo l'utente può richiedere le credenziali.

Successivamente, l'Autorità genera un Merkle Tree dove delle foglie rappresentano la credenziale (tipo credenziale, valore credenziale, data di scadenza) e una la chiave pubblica dell'utente. Per ogni foglia del Merkle Tree, viene calcolato un commitment utilizzando una funzione hash resistente, SHA-256, che rappresenta un'impronta univoca della foglia.

Dopo aver creato il Merkle Tree con i commitment delle foglie, l'Autorità calcola la radice del Merkle tree, e la firma digitale utilizzando la propria chiave privata.

L'Autorità rilascia all'utente:

- La credenziale (specificato più avanti).
- La firma digitale della radice del Merkle Tree.
- Il proprio certificato, utile per richiedere l'accesso ai servizi.

In questo modo, l'utente riceve una credenziale con il commitment della foglia, che può essere utilizzato per verificare l'integrità della credenziale stessa attraverso il Merkle Tree. Inoltre, la firma digitale della radice del Merkle Tree fornisce un'ulteriore garanzia dell'autenticità del Merkle Tree e delle credenziali associate.

Credenziali

La credenziale generata dallo script è composta da tre campi fondamentali:

- **Tipo di credenziale**: rappresenta la natura della credenziale richiesta (ad esempio, "ISEE").
- **Valore della credenziale**: specifica il valore associato alla credenziale (ad esempio, "2500").
- **Data di scadenza**: indica la data di validità della credenziale (ad esempio, "2026").

Questa credenziale è concatenata in un'unica stringa con i campi separati da un punto e virgola (ad esempio: “ISEE;2500;2026;”)

2.4 Accesso ai servizi

Una volta stabilita una connessione sicura tra utente e fornitore di servizi tramite TLS, l’utente procede con il richiedere i servizi del server inviando il proprio certificato (CIE), il certificato dell’autorità che ha rilasciato la credenziale, la credenziale e la root firmata. Il server non accede a tutti i campi del certificato dell’utente ma soltanto al campo relativo alla chiave pubblica.

Il server, ricevuta la richiesta, si calcola la root del Merkle Tree (Root_MT) a partire dalla chiave pubblica, estrapolata dal certificato utente, e dalla credenziale ricevuta. In seguito, se riconosce come affidabile le autorità che hanno rilasciato le credenziali possedute da U effettua una verifica tramite l’algoritmo $Vrfy_{pksign}$ (Root_MT, σ) per riconoscere l’autenticità delle credenziali.

Tramite questa sequenza di operazioni il server è ora in possesso della chiave pubblica dell’utente; può quindi fare affidamento al fatto che l’utente conosca un’informazione segreta (sk) correlata a tale chiave. L’esito della verifica viene comunicato all’utente e in caso esso sia positivo l’utente ha accesso ai servizi qualificati.

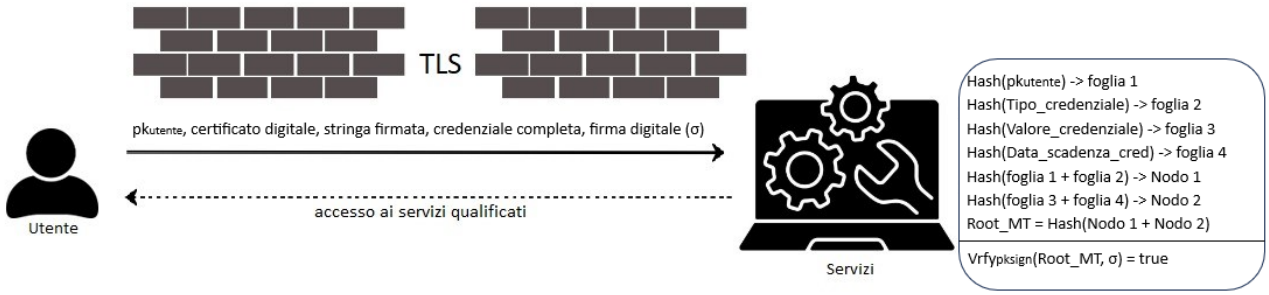

Figura 2: Accesso ai servizi

Alcuni servizi richiedono una o più credenziali rilasciate da una o più autorità ma associate tutte a un singolo utente. Tutte le credenziali da combinare devono essere rilasciate da una o più autorità certificate dall’IPZS.

2.5 Specifiche utilizzate

2.4.1 Certificato digitale

Un certificato digitale è un documento elettronico che utilizza la crittografia per autenticare l'identità delle parti coinvolte in una comunicazione digitale e per garantire la sicurezza delle informazioni scambiate. Un certificato digitale contiene una chiave pubblica e l'identità del proprietario della chiave pubblica. È rilasciato da un'entità fidata nota come Certification Authority (CA)

Il formato da noi utilizzato è il X509v3

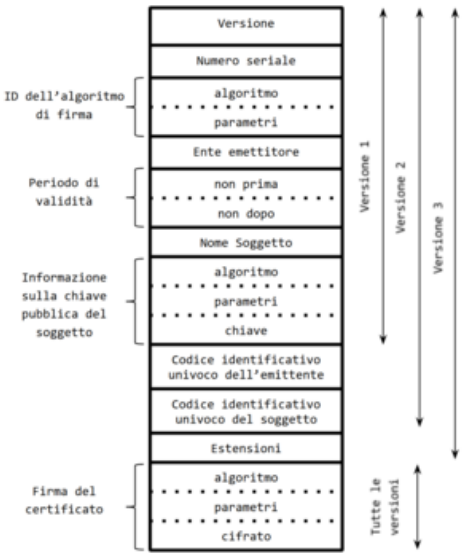

Figura 3: Certificato digitale X509v3

2.4.2 SHA-256

SHA-256 è un algoritmo di hash (Secure Hash Algorithm) a 256 bit che viene utilizzato per la sicurezza crittografica. Sinteticamente, le caratteristiche principali sono:

- Gli algoritmi di hash crittografici producono hash irreversibili e unici.
- Maggiore è il numero di possibili hash, minore è la possibilità che due valori creino lo stesso hash.

2.4.3 Firma ECDSA

L'ECDSA (Elliptic Curve Digital Signature Algorithm) è uno schema di firma digitale

Sinteticamente ECDSA funziona in questo modo:

- La firma codifica un punto casuale R (rappresentato solo dalla sua coordinata x) tramite trasformazioni di curve ellittiche usando la chiave privata privKey e l'hash del messaggio h in un numero s, che è la prova che il firmatario del messaggio conosce la chiave privata privKey. La firma { r , s } non può rivelare la chiave privata.
- La verifica della firma decodifica il numero di prova s dalla firma al suo punto originale R , utilizzando la chiave pubblica pubKey e l'hash del messaggio h e confronta la coordinata x del R recuperato con il valore r dalla firma.

2.4.4 Merkle tree

Un Merkle Tree, o albero di Merkle, è una struttura dati ad albero in cui ogni foglia contiene un hash crittografico di un blocco di dati e serve ad avere una rappresentazione compatta di più elementi. Gli hash delle foglie vengono quindi combinati in modo ricorsivo fino a ottenere un singolo hash noto come radice del Merkle Tree (Merkle root). Questa radice rappresenta l'integrità dell'intero set di dati contenuti nelle foglie.

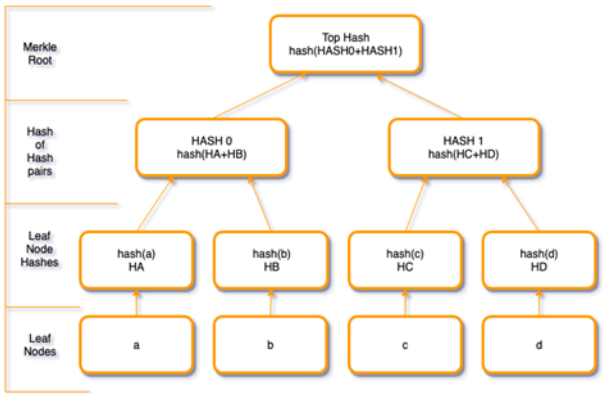

Figura 4 Creazione Merkle Tree

2.4.5 Commitment

Un commitment è un'impronta univoca e resistente alle collisioni di un dato, ottenuta attraverso una funzione hash crittografica. Nel contesto del Merkle Tree, il commitment viene calcolato per ogni foglia dell'albero.

Vantaggi dell'Utilizzo del Merkle Tree e del Commitment

Sicurezza Migliorata: La combinazione di Merkle Tree e commitment garantisce l'integrità e l'autenticità delle credenziali rilasciate.

Efficienza: La verifica della radice del Merkle Tree è computazionalmente efficiente e richiede solo una piccola quantità di dati aggiuntivi.

Resistenza alla Manomissione: Qualsiasi modifica non autorizzata ai dati di una foglia sarà rilevata grazie alla radice del Merkle Tree.

Scalabilità: Il Merkle Tree è adatto a gestire grandi quantità di dati, poiché la radice è di dimensioni fisse indipendentemente dal numero di foglie.

2.4.6 TLS

Gli scopi principali del protocollo TLS (Transport Layer Security) è di garantire privacy e integrità dei dati tra due applicazioni che comunicano. Il protocollo è composto da due livelli: TLS Record Protocol e TLS Handshake Protocol.

Al livello più basso il TLS si allaccia con un protocollo di trasporto, attraverso il TLS Record Protocol.

IL **TLS Record Protocol** è incapsulato in protocolli di livello più alto. Uno di questi protocolli è il TLS Handshake Protocol, che permette al server e al client di autenticarsi e negoziare un algoritmo di cifratura e le eventuali chiavi, prima che vengano trasmessi eventuali dati.

Il **TLS Handshake Protocol** dà garanzia di connessioni sicure che hanno tre proprietà basilari:

- L'identità del client e del server può essere autenticata usando la crittografia asimmetrica, o a chiavi pubbliche. Questa può essere opzionale, ma in genere è richiesta da almeno un peer.
- La negoziazione di un segreto condiviso è sicura: la negoziazione di segreti non deve permettere ad eventuali attaccanti di impersonare uno dei peer.
- La negoziazione è affidabile: nessun attaccante può modificare la comunicazione delle parti che effettuano la negoziazione.

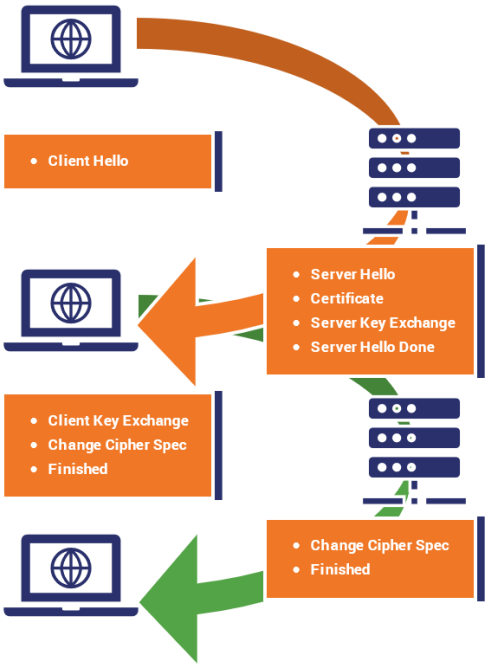

Figura 5:TLS

WP3

Questo work package ha lo scopo di analizzare la soluzione presentata in WP2 rispetto al modello presentato in WP1. Verranno esaminate le principali proprietà di sicurezza: confidenzialità, integrità, trasparenza ed efficienza. Alla fine del documento, verrà esibito e giustificato un grafico radar che rappresenta le prestazioni del sistema in base a queste quattro proprietà.

3.1 Confidenzialità

La confidenzialità è una proprietà cruciale per garantire che le informazioni sensibili degli utenti siano protette e accessibili solo dalle parti autorizzate.

- **C.1**: **Le credenziali degli utenti devono essere protette e accessibili solo dalle parti autorizzate in maniera controllata**.
  Il Merkle Tree garantisce che le credenziali siano protette da manomissioni. Il protocollo TLS, invece, assicura che le credenziali siano accessibili solo alle parti autorizzate, proteggendo il canale di comunicazione da interferenze e intercettazioni non autorizzate.

- **C.2**: **Le comunicazioni tra gli utenti e i servizi devono essere protette per garantire che nessuna informazione sensibile venga intercettata.**
  Le comunicazioni tra l'utente e il servizio sono protette e rese accessibili esclusivamente alle parti autorizzate grazie all'uso di connessioni TLS, che garantiscono la sicurezza nella trasmissione dei dati sensibili.

- **C.3**: **Le informazioni personali degli utenti devono essere minimizzate nelle comunicazioni per evitare esposizioni non necessarie.**
  L'utente invia il proprio certificato digitale sia all'autorità di rilascio delle credenziali sia all'ente per l'accesso ai servizi. L'autorità di rilascio ha accesso a tutti i dati presenti nel certificato, ma utilizza solo le informazioni necessarie per il rilascio delle credenziali. L'ente per l'accesso ai servizi, invece, ha accesso esclusivamente alla chiave pubblica dell'utente.

- **C.4: Il sistema deve essere progettato in modo che un utente malintenzionato non possa usufruire dei servizi disposti, identificandosi con le credenziali di un altro utente.**
  Le credenziali sono associate a un utente attraverso la sua chiave pubblica. Per accedere a un servizio, l'utente deve dimostrare di essere il legittimo proprietario della credenziale, provando di possedere la corrispondente chiave privata.

- **C.5**: **Le credenziali devono poter essere mostrate in maniera sicura e verificabile, garantendo che tali informazioni, una volta verificate per l'autenticazione, non vengano conservate o memorizzate dal server per qualsiasi scopo.**

Il server verifica le credenziali, ma non conserva le informazioni personali dopo l'autenticazione. Come supposto nel paragrafo 2.2 del WP2.

3.2 Integrità

L'integrità è fondamentale per garantire che i dati non siano stati alterati in modo non autorizzato e che le credenziali siano autentiche.

- **I.1**: **Le credenziali devono essere verificabili e non falsificabili, garantendo che siano state emesse da un'autorità riconosciuta.**
  Nel sistema proposto è utilizzato un Merkle Tree per garantire l'integrità e l'autenticità delle credenziali. La root dell'albero è firmata dell'autorità emittente, assicurando così l'autenticità e la verificabilità delle credenziali. Inoltre, qualsiasi modifica apportata a una parte delle credenziali risulterà incompatibile con la root dell'albero, rendendo impossibile la falsificazione o la modifica delle credenziali.

- **I.2**: **Le politiche di accesso devono essere applicate correttamente, impedendo l'accesso a chi non possiede le credenziali richieste.**
  Il sistema implementa controlli rigorosi per l'accesso ai servizi, verificando che le credenziali fornite dall'utente soddisfino i requisiti richiesti:
  - Le credenziali sono emesse dall'autorità di rilascio solo agli utenti che rispettano determinati requisiti (ad esempio, ISEE < 5000, luogo di nascita: Napoli, ecc.);
  - L'utente richiede l'accesso ai servizi del server inviando il proprio certificato (CIE), il certificato dell'autorità che ha emesso la credenziale, la credenziale stessa e la root firmata. Prima di consentire l'accesso ai servizi, il server verifica l'autenticità delle credenziali, assicurandosi che non siano state manomesse, e controlla che le credenziali siano state rilasciate da enti fidati, confrontando la firma della root con la chiave pubblica dell'autorità.

- **I.3: La chiave segreta della CIE deve rimanere nascosta e non accessibile a nessun attaccante.**
  L'utente dimostra di possedere la chiave privata senza condividerla con gli altri attori del sistema.

- **I.4**: **Il sistema deve garantire che le credenziali non possano essere combinate da utenti diversi per accedere a servizi riservati a chi possiede entrambe le credenziali.**
  Il sistema garantisce che le credenziali da combinare siano associate a un singolo utente, verificando l'identità attraverso la chiave pubblica presente nel certificato e quella associata al Merkle Tree.

- **I.5: Le informazioni scambiate tra gli utenti e il server devono mantenere la loro integrità durante tutto il processo di comunicazione, evitando modifiche non autorizzate.**
  La connessione TLS garantisce che tutte le informazioni scambiate tra l'utente e il server siano crittografate, proteggendo l'integrità dei dati durante la comunicazione.

3.3 Trasparenza

La trasparenza è essenziale per garantire che gli utenti e le autorità possano fidarsi del sistema.

Ecco le proprietà di trasparenza ordinate per livello di dettaglio e completezza:

- **T.1**: **Gli algoritmi e i protocolli utilizzati per la gestione delle credenziali e l'accesso ai servizi devono essere pubblicamente noti e verificabili.**
  Gli algoritmi e protocolli impiegati, come TLS, certificati X509v3, firme digitali e Merkle tree, sono standard pubblicamente noti e verificabili, garantendo una buona trasparenza tecnica.

- **T.2**: **Il processo di emissione, verifica e gestione delle credenziali deve essere trasparente e verificabile da parte degli utenti e delle autorità competenti.**
  La trasparenza e la verificabilità del processo di emissione, verifica e gestione delle credenziali sono fondamentali per assicurare fiducia nel sistema da parte degli utenti e delle autorità. Nel sistema proposto, questo processo si basa su tecnologie riconosciute e consolidate, come:
  - Certificati digitali: Le credenziali vengono emesse da un'autorità certificata e includono una firma digitale che permette di verificare l'autenticità e l'integrità della credenziale.
  - Merkle Tree: Le credenziali sono strutturate in un Merkle Tree, che garantisce l'integrità del set di dati. La radice del Merkle Tree, firmata dall'autorità di emissione, consente la verifica dell'autenticità delle credenziali in modo trasparente, riducendo il carico di verifica senza compromettere la sicurezza.
  - Firme digitali: L'uso di firme digitali nella radice del Merkle Tree permette di garantire che la credenziale emessa non sia stata alterata, e la firma può essere verificata da qualsiasi utente autorità che possieda la chiave pubblica dell'autorità di emissione.

  Tutti questi meccanismi forniscono il soddisfacimento della proprietà.

- **T.3**: **Minimizzare l'affidamento su terze parti fidate, garantendo che il sistema sia progettato per essere verificabile indipendentemente.**
  Il sistema è progettato per minimizzare l'affidamento su terze parti fidate attraverso l'implementazione di meccanismi di verifica indipendenti e architetture decentralizzate. La combinazione di Merkle tree, algoritmi pubblici, e procedure trasparenti crea un ambiente di fiducia e sicurezza, dove gli utenti possono autonomamente confermare l'integrità e l'autenticità delle credenziali senza dipendere completamente da entità esterne.

3.3 Efficienza/Usabilità

L'efficienza e l'usabilità sono importanti per garantire che il sistema sia pratico e possa essere utilizzato senza problemi dagli utenti.

- **E.1**: **Il sistema deve garantire tempi di risposta rapidi per la verifica delle credenziali e l'accesso ai servizi.**
  La soluzione proposta utilizza il Merkle Tree e il commitment, risultando particolarmente efficiente per la verifica delle credenziali. Il Merkle Tree consente di verificare l'integrità di un insieme di dati con un overhead computazionale ridotto, poiché solo una piccola parte dei dati (le foglie) deve essere processata per calcolare la radice del Merkle Tree. Inoltre, la firma digitale della radice garantisce un controllo rapido dell'autenticità delle credenziali, migliorando ulteriormente i tempi di verifica.
  Il sistema impiega anche l'uso di TLS per stabilire connessioni, ottimizzato per garantire comunicazioni rapide e sicure. Questo assicura che la fase di connessione per l'autenticazione e l'accesso ai servizi sia veloce.

- **E.2: Il processo di autenticazione dell'utente, tramite CIE deve essere rapido e sicuro, minimizzando i tempi di attesa per l'utente.**
  Il processo di autenticazione dell'utente è rapido e sicuro grazie all'uso di ECDSA per la firma e verifica digitale. Il processo descritto prevede che l'utente fornisca il PIN e che la CIE esegua una firma ECDSA del messaggio. Questo meccanismo è rapido, minimizzando così i tempi di elaborazione.

- **E.3: L'autorità di credenziali deve garantire che il rilascio della credenziale sia eseguita in modo rapido ed efficiente, riducendo al minimo l'impatto sulle prestazioni del sistema e l'esperienza degli utenti.**
  Rilasciare le credenziali, assieme alla firma digitale della radice del Merkle tree, è una soluzione che riduce il carico di lavoro e quindi riduce l'impatto sulle prestazioni. La firma della radice garantisce l'autenticità dell'intero insieme di dati, evitando la necessità di firmare individualmente ogni credenziale.

3.4 Valutazioni

L'analisi delle proprietà di confidenzialità, integrità, trasparenza ed efficienza/usabilità ha messo in luce i punti di forza e le aree di miglioramento del sistema esaminato. Di seguito, una valutazione approfondita di ciascuna proprietà, con punteggi assegnati da 1 a 100 per riflettere la qualità complessiva della soluzione.

Valutazione confidenzialità (94/100):

La soluzione proposta garantisce un'eccellente protezione delle informazioni sensibili e una gestione sicura dell'accesso e delle comunicazioni.

Tuttavia, alcuni miglioramenti sono possibili nella minimizzazione delle informazioni trasmesse nelle comunicazioni tra l'utente e l'autorità di rilascio delle credenziali, per ridurre ulteriormente l'esposizione dei dati. Un'altra area di miglioramento riguarda la gestione delle modalità di cancellazione delle informazioni dopo l'uso, per prevenire conservazioni involontarie che potrebbero compromettere la confidenzialità a lungo termine.

Valutazione integrità (100/100):

L'integrità del sistema è garantita in modo ottimale attraverso l'utilizzo efficace di Merkle Tree e firme digitali per garantire autenticità e verifica rapida delle credenziali; inoltre, la protezione è robusta contro le modifiche non autorizzate.

Valutazione trasparenza (93/100):

Il sistema proposto presenta un robusto approccio alla trasparenza, evidenziato dall'uso di tecnologie consolidate e standardizzate. La chiara documentazione degli algoritmi e dei protocolli, insieme a un processo di emissione e gestione delle credenziali ben definito, contribuisce a garantire fiducia da parte degli utenti e delle autorità competenti. La minimizzazione dell'affidamento su terze parti fidate, tramite meccanismi di verifica indipendenti come i Merkle Tree e le firme digitali, rappresenta un ulteriore passo avanti verso un sistema più sicuro e autonomo.

La presenza di molteplici tecnologie e protocolli può rendere difficile comprendere appieno il funzionamento del sistema.

In generale, la progettazione del sistema favorisce una maggiore trasparenza e responsabilità, elementi fondamentali in contesti in cui la sicurezza e la privacy degli utenti sono prioritari.

Valutazione efficienza/usabilità (88/100):

Il sistema dimostra un buon livello di efficienza, grazie all'uso del Merkle Tree che ottimizza la verifica delle credenziali, e alla firma digitale della root, che velocizza il controllo dell'autenticità. Il processo di autenticazione tramite CIE è rapido e sicuro, supportato dall'uso di ECDSA per la firma e verifica. Tuttavia, il rilascio delle credenziali potrebbe essere ulteriormente migliorato, tenendo conto delle conseguenze sull’efficienza della verifica delle credenziali.

Valutazione generale del sistema (94/100):

Nel complesso, il sistema presenta un ottimo equilibrio, con punteggi elevati nelle proprietà di integrità e confidenzialità e trasparenza, mentre potrebbe beneficiare di miglioramenti in termini di ottimizzazione dell'efficienza nei processi di rilascio. Il grafico radar conclusivo rifletterà questo

bilanciamento positivo, con punte leggermente inferiori per l'efficienza rispetto alle altre proprietà.

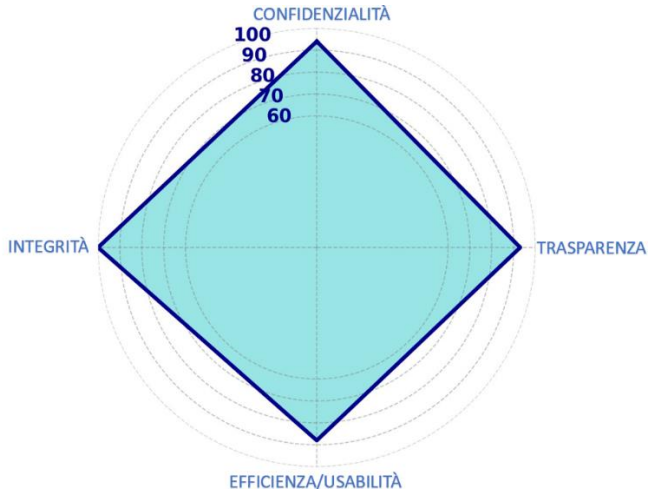

Figura 6: grafico radar

3.5 Contrasto degli avversari

I possibili avversari da sconfiggere si dividono in due tipologie:

- attivi: è un avversario "semi-onesto" oppure "onesto ma curioso". Egli effettua il compito previsto dal sistema/protocollo in cui si trova ma colleziona dati (che non gli spettano), perché vuole provare a dedurre qualcosa.
- passivi: è un avversario "completamente disonesto" che devia le specifiche del protocollo, ovvero non segue le regole del protocollo. Il suo obiettivo è rubare dati che non gli spettano o addirittura modificare dati, rompendo la confidenzialità e/o l'integrità del sistema.

Un avversario attivo corre più rischi, perché può essere facilmente scoperto e punito dal sistema che sta cercando di deviare. Quindi, scoraggiato da ciò, è più raro di uno passivo. Un avversario passivo, invece, è più difficile da identificare e punire.

Detto ciò, è utile considerare alcuni avversari passivi per l'analisi perché un qualsiasi attore, anche fidato, potrebbe fungere da avversario passivo. Un sistema efficace oltre che contrastare gli avversari attivi deve scoraggiare gli avversari passivi a diventare attivi.

Impersonator

Avversario Attivo

L'Impersonator è l'avversario che cerca di impersonificare un utente legittimo utilizzando credenziali rubate o falsificate. Questo tipo di attacco è contrastato attraverso:

- **TLS (Transport Layer Security)**: Impedisce il furto delle credenziali attraverso un canale sicuro, garantendo la sicurezza delle comunicazioni, impedendo intercettazioni e accessi non autorizzati.
- Se l'avversario riuscisse ad ottenere le credenziali in altro modo, non sarebbe comunque in grado di utilizzarle per i propri scopi, poiché non potrebbe dimostrare di essere il legittimo possessore della credenziale (grazie all'utilizzo della chiave pubblica).

Questa tipologia di avversario può essere contrastata affinché l'Impersonator non entra in possesso della chiave privata dell'utente.

Malicious Autority

Avversario Attivo

Il Malicious Authority è un avversario che tenta di emettere credenziali false o alterate.

Un'autorità per essere considerata affidabile deve possedere un certificato firmato dall'IPZS. Questa tipologia di avversario ha un certificato non valido e quindi grazie all'uso del protocollo TLS, l'autorità malevola non può stabilire una connessione sicura con l'utente, impedendo così l'emissione di credenziali.

Un caso diverso si verifica quando l'utente si affida volontariamente a una Malicious Authority perché non soddisfa i requisiti per il rilascio di credenziali. Anche in questo scenario, il certificato digitale firmato dall'IPZS risulta cruciale: quando l'utente cerca di accedere a un servizio, sarà obbligato a mostrare il certificato dell'autorità che ha emesso le credenziali. Tuttavia, l'accesso al servizio sarà negato, poiché l'ente non riconoscerà come valide le credenziali emesse da un'autorità non affidabile.

Nota: In questo contesto, non viene impedito il rilascio delle credenziali false, ma viene bloccato il loro utilizzo.

Per quanto riguarda la manipolazione dei dati, l'uso del Merkle Tree consente di rilevare eventuali alterazioni sulle credenziali.

Dinosaurs

Avversario Passivo

I Dinosaurs sono oppositori dell'innovazione che mirano a dimostrare falle nella sicurezza e nella privacy. Vengono contrastati grazie all'uso delle seguenti contromisure:

- **Trasparenza del sistema:** utilizzo di protocolli di sicurezza standard come TLS e X509v3 per garantire la riservatezza e la protezione delle comunicazioni. La trasparenza nell'uso di queste tecnologie, già ampiamente riconosciute per la loro sicurezza, aiuta a contrastare le campagne di disinformazione, fornendo solide garanzie sul funzionamento del sistema.
- **Utilizzo di strumenti crittografici:** strumenti come la firma digitale e i Merkle Tree sono tecnologie crittografiche avanzate ampiamente adottate per garantire l'integrità e l'autenticità dei dati. L'integrazione di queste tecniche rafforza la solidità del sistema,

dimostrando che esso si basa su meccanismi di sicurezza robusti e riconosciuti nel campo della crittografia.

The Eavesdropper

Avversario Passivo

The Eavesdropper è l'avversario interessato a tutte le operazioni del sistema, in particolare si concentra nell'intercettare le comunicazioni e analizzare il traffico. La minimizzazione delle informazioni trasmesse riduce la quantità di dati sensibili che potrebbero essere intercettati. Ma l'avversario viene contrastato dall'uso della **connessione TLS**: L'intero processo di autenticazione e rilascio delle credenziali avviene attraverso una connessione crittografata TLS. Questo impedisce a eventuali intercettatori di accedere ai dati sensibili scambiati tra l'utente e il server.

Internal Opponent

Avversario Attivo

L' Intenal opponent è l'avversario interno a un sistema che potrebbe tentare di acquisire informazioni personali degli utenti in modo improprio.

Abbiamo due tipologie di sistemi:

- Rilascio credenziali: Abbiamo supposto che un ente fidato abbia un certificato firmato IPZS, questo perché questa tipologia di enti hanno accesso a molte informazioni riservate e quindi devono garantire l'affidabilità.
- Accesso ai servizi: Abbiamo supposto che l'ente che fornisce l'accesso ai servizi è sempre un attore onesto e quando riceve le credenziali da un utente non le conserva. Inoltre ha accesso a un minimo di informazioni sull'utente, quali credenziali e certificato digitale per estrapolare la chiave pubblica.

Colluding Users

Avversario Attivo

I colluding users sono avversari che uniscono le credenziali per soddisfare i requisiti di accesso ai servizi. Questa tipologia di avversari viene contrastata da:

- **Verifica basata su Merkle Tree**: Ogni credenziale è univocamente legata all'identità dell'utente, grazie all'uso della chiave pubblica, tramite il sistema Merkle Tree, rendendo impossibile combinare credenziali diverse, per ottenere un accesso fraudolento, senza che venga rilevata una discrepanza.

User Requesting Corrupt Entity Credentials

Avversario Attivo

L' User Requesting Corrupt Entity Credentials è un ente corrotto, finge di essere un'autorità riconosciuta, che richiede le credenziali a un'autorità per un utente onesto.

Abbiamo tre attori: utente onesto, autorità disonesta, autorità fidata.

Un’autorità per essere considerata affidabile deve possedere un certificato firmato dall’IPZS. Questa tipologia di avversario ha un certificato non valido e quindi grazie all'uso del protocollo TLS, l'autorità malevola non può stabilire una connessione sicura con l'utente.

Expired ID User

Avversario Attivo

L'Expired ID User è un avversario che tenta di utilizzare una carta d'identità scaduta per ottenere delle credenziali. Il certificato digitale X509v3 contiene i campi che specificano l'inizio e la fine del periodo di validità.

Ciò implica che, durante i controlli effettuati dal server per autenticare l'utente e rilasciare le credenziali, se il certificato risultasse scaduto e quindi non valido, il server negherebbe l'autenticazione.

Conclusioni

La soluzione proposta nel WP2 adotta misure rigorose di autenticazione, crittografia e verifica per proteggere il sistema da una vasta gamma di avversari. Grazie all'uso di tecnologie consolidate come la CIE, il Merkle Tree e i certificati digitali, il sistema garantisce che solo utenti autorizzati possano accedere ai servizi, prevenendo efficacemente attacchi sia attivi che passivi.

WP4

L'obiettivo del WP4 è implementare le funzionalità del sistema descritte nel WP2, utilizzando comandi OpenSSL per operazioni crittografiche come la generazione di chiavi, firme digitali, calcolo di hash e commitment. Un elemento chiave è la creazione del Merkle tree, in cui ogni foglia rappresenta una credenziale o una chiave pubblica. L'Autorità emittente calcola e firma digitalmente la radice del Merkle tree, garantendo l'integrità e l'autenticità dei dati.

Gli script forniti sono:

- init.sh
- certificato_utente.sh
- certificato_autorità.sh
- certificato_autorità2.sh
- richiesta_rilascio_credenziali.sh
- richiesta_rilascio_credenziali2.sh
- accesso_servizio.sh
- accesso_servizio_2cred.sh

Gli script che contengono il numero "2" nel nome indicano che sono destinati all'accesso ai servizi tramite la combinazione di due credenziali valide.

Per accedere a un servizio con una singola credenziale, l’ordine di esecuzione degli script è il seguente:

1. init.sh
2. certificato_utente.sh
3. certificato_autorità.sh
4. richiesta_rilascio_credenziali.sh
5. accesso_servizio.sh

Invece, per accedere a un servizio con la combinazione di credenziali, l’ordine di esecuzione degli script è il seguente:

1. init.sh
2. certificato_utente.sh
3. certificato_autorità.sh
4. certificato_autorità2.sh
5. richiesta_rilascio_credenziali.sh
6. richiesta_rilascio_credenziali2.sh
7. accesso_servizio_2cred.sh

Nota: certificato_autorità.sh e certificato_autorità2.sh possono essere invertiti e anche richiesta_rilascio_credenziali.sh conrichiesta_rilascio_credenziali2.sh

4.1 Init

Lo script bash presentato è progettato per inizializzare un ambiente strutturato per la gestione delle directory necessarie alla configurazione e operatività delle autorità e degli utenti nel sistema. In particolare, crea le cartelle per l'Istituto Poligrafico e Zecca dello Stato (IPZS), per l'Autorità di Credenziale (Aut1), e per l'utente. Questa preparazione è essenziale per garantire una corretta organizzazione delle risorse, facilitando la successiva gestione delle credenziali e delle chiavi pubbliche.

Definizione delle cartelle:

- **`IPZS_dir="IPZS":`** Definisce il nome della directory per i dati dell'Istituto Poligrafico e Zecca dello Stato.
- **`Aut1_dir="Aut1":`** Definisce il nome della directory per l'autorità riconosciuta che rilascia le credenziali.
- **`Utente_dir="Utente":`** Definisce il nome della directory per i dati dell'utente.

Creazione delle cartelle:

- **`mkdir -p $IPZS_dir`**: Crea la directory per IPZS se non esiste già. L'opzione -p permette di creare anche eventuali directory parentali mancanti.
- **`mkdir -p $Aut1_dir`**: Crea la directory per l'autorità di rilascio delle credenziali se non esiste già.
- **`mkdir -p $Utente_dir`**: Crea la directory per l'utente se non esiste già.

Messaggio di conferma:

- **`echo "Le directory sono state create correttamente"`**: Mostra un messaggio di conferma che le directory sono state create con successo.

4.2 certificato_utente

Lo script presentato è progettato per simulare il processo di creazione e firma di certificati digitali per un utente utilizzando l'Istituto Poligrafico e Zecca dello Stato (IPZS) come Certification Authority (CA). Questo script dimostra come l'IPZS genera il proprio certificato autofirmato e successivamente utilizza questo certificato per firmare il certificato dell'utente, assicurando così che l'utente possa essere autenticato in modo sicuro nel sistema.

Definizione delle cartelle:

- **`IPZS_dir="IPZS":`** Definisce il nome della directory per i dati dell'Istituto Poligrafico e Zecca dello Stato.

- **Utente_dir="Utente":** Definisce il nome della directory per i dati dell'utente.

Generazione della Chiave Privata per IPZS:

- Utilizzando **openssl ecparam -genkey -name prime256v1**, lo script genera una chiave privata per IPZS con l'algoritmo di crittografia a curve ellittiche (ECC) usando la curva `prime256v1`. Questa chiave è fondamentale per creare e firmare certificati digitali.

Creazione del Certificato Autofirmato per IPZS:

- Con **`openssl req -x509 -new -key $IPZS_sk`**, lo script crea un certificato autofirmato per IPZS che servirà da CA. Questo certificato contiene informazioni come il nome del soggetto e la chiave pubblica derivata dalla chiave privata di IPZS. L'autofirma attesta che IPZS è una CA fidata.

Generazione della Chiave Privata per l'Utente:

- Lo script genera una chiave privata per l'utente, anch'essa utilizzando ECC con la curva **`prime256v1`**. Questa chiave sarà utilizzata dall'utente per firmare dati e autenticarsi nel sistema.

Creazione della Richiesta di Certificato (CSR) per l'Utente:

- Utilizzando **openssl req -new**, lo script crea una richiesta di certificato (CSR) per l'utente. Questa richiesta contiene la chiave pubblica dell'utente e altre informazioni necessarie per la creazione del certificato, ma non è ancora firmata.

Firma del Certificato dell'Utente da Parte della CA IPZS:

- Lo script utilizza **`openssl x509 -req`** per firmare la CSR dell'utente con la chiave privata di IPZS, generando così un certificato per l'utente che è ufficialmente riconosciuto dalla CA IPZS. Questa firma lega l'identità dell'utente alla sua chiave pubblica in modo verificabile.

Verifica del Certificato dell'Utente:

- Infine, lo script esegue **`openssl verify -CAfile $IPZS_crt $Utente_cert`** per verificare che il certificato dell'utente sia stato correttamente firmato da IPZS. Questa verifica conferma che il certificato dell'utente è autentico e affidabile.

4.3 certificato_autorità

In questa sezione, descriviamo il processo di creazione, richiesta e verifica di un certificato per l'Autorità di Credenziali (Aut1) all'interno del sistema, firmato da un'autorità di certificazione fidata come IPZS. Il certificato di Aut1 è essenziale per garantire l'autenticità e l'affidabilità delle operazioni dell'Autorità di Credenziali, che ha il compito di rilasciare credenziali sicure agli utenti.

Lo script certificato_autorità.sh automatizza il processo, coprendo la generazione della chiave privata, la creazione della richiesta di certificato (CSR), la firma del certificato e la sua verifica finale. Di seguito vengono illustrati i principali passaggi.

Generazione della Chiave Privata per l'Autorità 1:

- Utilizza il comando **`openssl ecparam -genkey`** con la curva **`prime256v1`** per generare una chiave privata per Aut1. Questa chiave è essenziale per firmare le richieste di certificato e per operare in modo sicuro come Autorità di Credenziali.

Creazione della Richiesta di Certificato (CSR) per Aut1:

- Lo script crea una richiesta di certificato (CSR) per Aut1 utilizzando la chiave privata generata, con il comando **`openssl req -new`**. Il CSR è necessario per richiedere formalmente un certificato da una CA (in questo caso, IPZS), contiene la chiave pubblica di Aut1 e altre informazioni che definiscono l'identità dell'autorità.

Firma del Certificato di Aut1 da Parte della CA IPZS:

- Utilizza **`openssl x509 -req`** per firmare il CSR di Aut1 con il certificato e la chiave privata di IPZS. Questo crea un certificato per Aut1 che è ufficialmente riconosciuto da IPZS, che conferma l'identità di Aut1 come un'autorità affidabile. La firma del certificato di Aut1 da parte di una Certification Authority fidata come IPZS è essenziale perché assicura che il certificato sia autentico e che Aut1 possa operare in modo credibile all'interno del sistema. Questa relazione di fiducia tra IPZS e Aut1 è cruciale per mantenere l'integrità e la sicurezza del sistema di gestione delle credenziali, garantendo che solo entità autorizzate possano rilasciare credenziali valide.

Verifica del Certificato di Aut1:

- Lo script verifica il certificato di Aut1 usando **`openssl verify`**, assicurandosi che sia stato correttamente firmato da IPZS. Tale verify conferma che il certificato sia autentico e che tutte le informazioni siano state firmate correttamente dalla CA. Questo step aiuta a prevenire l'uso di certificati falsificati o non autorizzati all'interno del sistema.

4.4 certificato_autorità2

Lo script certificato_autorità2.sh è progettato per generare e gestire le chiavi e i certificati digitali di una seconda Autorità di Credenziali, denominata Aut2, utilizzando la Certification Authority (CA) IPZS. L'obiettivo principale è creare una gerarchia di fiducia, in cui la CA IPZS firma i certificati delle autorità subordinate come Aut2, stabilendo così una catena di fiducia che consente di verificare le credenziali emesse da queste autorità. Firmando il certificato di Aut2, la CA IPZS assicura che solo entità riconosciute e autorizzate possano emettere credenziali, prevenendo così la possibilità che attori non autorizzati possano presentare credenziali falsificate.

4.5 richiesta_rilascio_credenziali

Lo script richiesta_rilascio_credenziali.sh simula un processo di richiesta e rilascio di credenziali digitali tra un utente e un'autorità di certificazione, utilizzando una Public Key Infrastructure (PKI). Il processo coinvolge la verifica dell'identità dell'utente, la firma di una stringa casuale per dimostrare il possesso della chiave privata e la successiva emissione di credenziali da parte dell'autorità.

L'interazione avviene in un ambiente sicuro (simulato con TLS), e la credenziale rilasciata è protetta utilizzando un Merkle tree, firmato dall'autorità. Di seguito, viene fornita una descrizione dettagliata di ciascun passaggio.

Inizializzazione delle Directory e File:

Lo script inizia definendo le directory in cui verranno salvati i file dell'utente, dell'autorità e della CA (IPZS). Successivamente, vengono specificati i file che saranno utilizzati o generati nel corso dell'esecuzione, come le chiavi private e pubbliche, le richieste di credenziali e la firma della stringa casuale.

Lato utente:

Identificazione dell'Utente con la CIE:

- Lo script inizia simulando l'inserimento del PIN della CIE da parte dell'utente. Il PIN inserito viene confrontato con il valore corretto (123456789). Se il PIN è corretto, l'autenticazione procede; in caso contrario, lo script termina con un messaggio di errore. Questa azione rappresenta l'autenticazione iniziale dell'utente tramite la propria CIE, confermando che l'utente è il legittimo proprietario della carta.

Definizione del Tipo e del Valore della Credenziale:

- L'utente definisce il tipo di credenziale richiesta (ad esempio, "ISEE") e il suo valore ("2500"). Queste informazioni vengono salvate rispettivamente nei file Tipo_credenziale e Valore_credenziale. Questi file contengono le specifiche della credenziale che l'utente desidera ottenere.

Creazione della Richiesta di Credenziale:

- Lo script genera una richiesta per una specifica credenziale, inserendo il tipo e il valore della credenziale in un file chiamato **`$Richiesta`**. Questo file contiene i dettagli della credenziale che l'utente desidera ottenere, formattati in modo strutturato per essere inviati all'autorità.

Firma di una Stringa Casuale:

- L'utente riceve dall'autorità una stringa casuale (ad esempio, "prova"), che deve essere firmata per dimostrare la proprietà della CIE. Questa stringa rappresenta una sfida che l'utente deve superare firmandola con la propria chiave privata.
- Lo script utilizza il comando **openssl dgst -sha256 -sign** per firmare la stringa casuale, e il risultato della firma viene salvato nel file **$Stringa_firmata**. Questa firma serve come prova crittografica del possesso della CIE.

Invio della Richiesta di Credenziale all'Autorità:

- Infine, l'utente invia la richiesta di credenziale all'autorità, insieme alla stringa firmata e al proprio certificato digitale. Questo passaggio è simulato nello script mostrando il contenuto del file **$Richiesta**, che contiene le informazioni essenziali per il rilascio della credenziale, compresa la prova della firma. Questa comunicazione permette all'autorità di verificare l'identità dell'utente e di procedere con il rilascio delle credenziali richieste.

Lato Autorità:

Ricezione della Richiesta:

- L'autorità riceve la richiesta di credenziale dall'utente, che include la richiesta stessa, il certificato digitale dell'utente, e la stringa firmata. Questa stringa firmata è utilizzata dall'autorità per verificare l'identità dell'utente e confermare che la richiesta provenga dal legittimo possessore della CIE.

Verifica del Certificato dell'Utente:

- Utilizza **`openssl verify`** con il certificato della CA IPZS per verificare che il certificato dell'utente sia autentico e sia stato correttamente firmato da IPZS. Questo passaggio è cruciale per confermare che il certificato sia stato correttamente emesso e firmato da IPZS, prevenendo l'uso di certificati falsificati e garantendo che l'identità dell'utente sia valida.

Estrazione della Chiave Pubblica dell'Utente:

- Estrae la chiave pubblica dal certificato digitale dell'utente utilizzando **`openssl x509 -in $Utente_cert -pubkey -noout -out $Utente_pk`**. La chiave pubblica è necessaria per verificare la firma della stringa casuale fornita dall'autorità.

Verifica della Firma sulla Stringa Casuale:

- Verifica la firma della stringa casuale con **`openssl dgst -sha256 -verify`**, utilizzando la chiave pubblica dell'utente. Se la verifica ha successo, conferma che l'utente possiede la chiave privata associata al certificato, dimostrando la propria identità in modo sicuro. Se la verifica fallisce, lo script termina con un messaggio di errore, impedendo il rilascio di credenziali a un utente non legittimo.

Controllo dei Requisiti per le Credenziali:

- L'autorità controlla se l'utente soddisfa i requisiti necessari per ottenere le credenziali, in questo caso un ISEE inferiore a 5000. Si suppone che i requisiti siano soddisfatti e l'autorità procede al rilascio delle credenziali.

Creazione della Credenziale:

- L'autorità crea la credenziale per l'utente, specificando la data di scadenza come "2026". La credenziale include le informazioni richieste, come il tipo di credenziale ("ISEE") e il suo valore ("2500"), assicurando che siano conformi alla richiesta specificata dall'utente e siano valide fino alla data indicata.

Creazione del Merkle Tree:

- Lo script crea un Merkle Tree per garantire l'integrità delle credenziali:
    - Calcola gli hash delle foglie per ogni componente della credenziale: chiave pubblica dell'utente, tipo di credenziale, valore della credenziale e data di scadenza.
    - Combina questi hash per formare i nodi intermedi e infine calcola la radice del Merkle Tree (**`$hash_root`**), che rappresenta un'impronta crittografica unica che riflette l'integrità e l'autenticità di tutte le informazioni incluse nel Merkle tree.

Rilascio delle Credenziali all'Utente:

- L'autorità rilascia all'utente la credenziale completa, la firma della radice del Merkle Tree (**$Root_firmata**), e il certificato dell'autorità di rilascio (**$Aut1_crt**). Questi elementi sono combinati in un file (**`$Cred_finale`**) che contiene tutte le informazioni necessarie per l'utente per dimostrare la validità delle sue credenziali.
- La firma della radice del Merkle tree con la chiave privata dell'autorità (**$Aut1_sk**) garantisce che la credenziale non sia alterata e provenga da una fonte legittima, rendendo le credenziali sicure e verificabili da terze parti.

Pulizia dei Dati Temporanei:

- Lo script rimuove i file temporanei utilizzati per il calcolo degli hash e la firma, come `Utente_pk.bin`, `Tipo_credenziale.bin`, `Valore_credenziale.bin`, e altri. Questo passaggio è importante per mantenere l'ambiente pulito e prevenire l'accumulo di dati inutili.

4.6 richiesta_rilascio_credenziale2

Lo script simula il processo di richiesta e rilascio di una seconda credenziale da parte di un utente, utilizzando la propria Carta d'Identità Elettronica (CIE) e l'autorità di rilascio Aut2. L'utente inserisce il PIN della CIE, specifica il tipo di credenziale (in questo caso, "Luogo di nascita") e firma una stringa casuale per dimostrare la proprietà della chiave privata della CIE. L'autorità verifica la validità del certificato dell'utente, conferma la firma della stringa casuale, controlla i requisiti per la credenziale, e crea un Merkle tree per garantire l'integrità dei dati. Infine, l'autorità firma la radice del Merkle tree e rilascia la credenziale, insieme alla firma e al certificato digitale dell'autorità. Grazie a questo script, l'utente può richiedere ulteriori credenziali con informazioni differenti, assicurando che ogni richiesta sia verificata e trattata separatamente per garantire sicurezza e integrità in ogni processo di rilascio.

4.7 accesso_servizio

Lo script accesso_servizio.sh gestisce il processo di autenticazione e verifica delle credenziali di un utente che cerca di accedere a un servizio qualificato. Utilizzando la firma digitale, certificati e la struttura del Merkle tree, lo script assicura che solo gli utenti legittimi con credenziali valide e non scadute possano accedere. Il sistema protegge l'integrità e la confidenzialità delle credenziali durante la comunicazione tra l'utente e l'autorità tramite TLS.

Inizializzazione delle Directory e File:

Lo script inizia definendo le directory in cui verranno salvati i file relativi all'utente, all'Autorità di Credenziali (Aut1) e alla Certification Authority (IPZS). Queste directory organizzano i file necessari per le operazioni di autenticazione e verifica, assicurando una chiara separazione tra i dati di ciascuna entità. Successivamente, lo script specifica i file che saranno utilizzati o generati nel corso dell'esecuzione, tra cui le chiavi private e pubbliche dell'utente e dell'autorità, i certificati digitali, la richiesta di credenziali, e le firme digitali. Questi file includono anche il certificato della CA IPZS, necessario per verificare la validità dei certificati rilasciati, la chiave pubblica estratta dal certificato dell'utente per la verifica delle firme, e la firma della root del Merkle tree che assicura l'integrità delle credenziali. La corretta gestione di questi file consente al sistema di mantenere un elevato livello di sicurezza e affidabilità durante l'autenticazione e l'autorizzazione degli utenti.

Lato utente:

Autenticazione tramite CIE:

- L'utente si autentica inserendo il PIN della sua Carta d'Identità Elettronica (CIE). Lo script verifica se il PIN inserito corrisponde al valore corretto (123456789). Se il PIN è corretto, l'autenticazione procede con successo; in caso contrario, lo script termina. Questo passaggio simula l'autenticazione iniziale basata sulla conferma che l'utente è il legittimo proprietario della CIE.

Firma della Stringa Casuale:

- L'utente riceve una stringa casuale dall'autorità e la firma utilizzando la propria chiave privata con il comando **`openssl dgst -sha256 -sign`**. Questa operazione serve a dimostrare il possesso della chiave privata associata al suo certificato digitale, confermando così la sua identità.

Invio al Server:

- L'utente invia al server la stringa firmata, il proprio certificato digitale, e le credenziali ottenute dall'autorità. Si assume che questa trasmissione avvenga su una connessione sicura (ad esempio TLS) per proteggere i dati durante il trasferimento.

Lato Server:

Ricezione e Verifica:

- Il server riceve la richiesta contenente la credenziale, il certificato digitale dell'utente e la stringa firmata. Inizia il processo di verifica controllando il certificato digitale dell'utente con il comando **openssl verify**, utilizzando il certificato della CA IPZS (**$IPZS_crt**). Questo passo assicura che il certificato dell'utente sia autentico e sia stato emesso da un'entità fidata, confermando così l'identità dell'utente.

Estrazione della Chiave Pubblica:

- Il server estrae la chiave pubblica dell'utente dal certificato ricevuto utilizzando **openssl x509 -in $Utente_cert -pubkey -noout -out $Utente_pk**. Questa chiave pubblica sarà utilizzata per verificare la firma della stringa casuale inviata dall'utente, dimostrando il possesso della chiave privata corrispondente.

Verifica della Firma sulla Stringa Casuale:

- Con **`openssl dgst -sha256 -verify`**, il server verifica che la firma sulla stringa casuale sia corretta utilizzando la chiave pubblica estratta dell'utente. Se la verifica ha successo, il server conferma che l'utente è il legittimo proprietario della chiave privata corrispondente al certificato, validando così l'identità dell'utente e l'integrità della richiesta.

Estrazione delle Credenziali:

- Lo script carica le credenziali dall'utente, che includono tipo, valore, e data di scadenza. Queste informazioni vengono elaborate per ulteriori verifiche.

Verifica della Data di Scadenza delle Credenziali:

- Lo script controlla che la data di scadenza delle credenziali sia successiva all'anno corrente (2024). Se la data è valida, il processo continua; altrimenti, le credenziali sono considerate scadute.

Verifica del Certificato dell'Autorità che ha Rilasciato le Credenziali:

- Il server verifica il certificato di Aut1, l'autorità che ha emesso le credenziali dell'utente, per assicurarsi che sia stato firmato da IPZS. Questa verifica conferma che Aut1 è una entità riconosciuta e fidata nel sistema.

Estrazione della Chiave Pubblica di Aut1:

- Dal certificato di Aut1, il server estrae la chiave pubblica, che verrà utilizzata per verificare la firma sulla root del Merkle tree delle credenziali.

Calcolo della Root del Merkle Tree:

- Il server ricostruisce la root del Merkle tree utilizzando le informazioni nelle credenziali: la chiave pubblica dell'utente, il tipo e il valore della credenziale, e la data di scadenza. Gli hash di queste informazioni sono combinati per formare i nodi del Merkle tree e infine calcolare la root.

Verifica della Firma della Root del Merkle Tree:

- Utilizzando la chiave pubblica di Aut1, il server verifica la firma sulla root del Merkle tree per confermare che le credenziali non siano state alterate. Se la verifica è positiva, le credenziali sono considerate autentiche e integre.

Pulizia dei Dati Temporanei:

- Infine, lo script elimina i file temporanei utilizzati durante il processo di verifica per mantenere l'ambiente pulito e prevenire l'accumulo di dati non necessari.

4.8 accesso_servizio_2cred

Questo script simula un processo di autenticazione e verifica di credenziali multiple per un utente che utilizza la propria Carta d'Identità Elettronica (CIE) interagendo con due diverse autorità di rilascio credenziali (Aut1 e Aut2). Il processo inizia con l'autenticazione dell'utente tramite l'inserimento del PIN corretto della CIE, seguito dalla firma di una stringa casuale per dimostrare il possesso della chiave privata associata alla CIE.

Il server riceve le credenziali e verifica la validità del certificato dell'utente e della firma sulla stringa casuale, confermando così l'identità dell'utente. Successivamente, il server estrae e verifica le informazioni delle due credenziali rilasciate dalle autorità Aut1 e Aut2, controllando le date di scadenza e la validità dei certificati delle autorità attraverso la CA IPZS. Per ogni credenziale, lo script calcola la radice del Merkle tree utilizzando gli hash delle informazioni rilevanti, tra cui la chiave pubblica, il tipo di credenziale, il valore e la data di scadenza, e verifica la firma digitale della radice fornita dalle autorità Aut1 e Aut2.

Questo processo garantisce l'integrità e l'autenticità delle credenziali dell'utente attraverso l'uso di firme digitali e strutture di dati sicure come il Merkle tree. Ogni verifica crittografica assicura che le informazioni non siano state alterate e che le credenziali siano state emesse da autorità riconosciute e fidate. Lo script viene utilizzato per simulare un ambiente sicuro di autenticazione e gestione delle

credenziali, verificando che ogni parte del processo sia protetta contro alterazioni e falsificazioni, assicurando così la sicurezza e l'integrità dell'intero sistema di gestione delle credenziali.